

Questions 41–60 — Full Solutions

Arrays (Easy & Medium) · In Depth · C++ & Python

What's inside. Items 41–60 — the heart of the **Arrays** step. The easy set (41–53) drills the fundamental moves (rotation, two-pointer, prefix-sum, XOR tricks); the medium set (54–60) is pure interview gold: **Two Sum, Dutch-flag Sort Colors, Boyer–Moore majority, Kadane's max-subarray, and Buy/Sell Stock**. Each solved in **both C++ and Python** with approach, full code, and complexity. All code compiled with `g++ -std=c++17` and run in Python.

Read the Approach, write both versions yourself, then compare. These patterns (two pointers, sliding window, prefix-sum + hashmap, single-pass tracking) reappear in hundreds of later problems.

Arrays — Easy (Q41–Q53)

Q41. Check if Array is Sorted

Easy

■ **Approach:** Walk once; if any element is greater than the next, it's not sorted (non-decreasing). Return false on the first violation, true if none.

C++

```
bool isSorted(vector<int>& a){
    for(int i = 0; i+1 < (int)a.size(); i++)
        if(a[i] > a[i+1]) return false;
    return true;
}
```

PYTHON

```
def is_sorted(a):
    return all(a[i] <= a[i+1]
              for i in range(len(a)-1))
```

Complexity & notes: $O(n)$ time, $O(1)$ space. 'Sorted' here means non-decreasing (equal neighbours are allowed).

Q42. Remove Duplicates from Sorted Array

Easy

■ **Approach:** Two pointers moving the same way: a WRITE pointer k for the next unique slot, a READ pointer i scanning. Since it's sorted, duplicates are adjacent — copy $a[i]$ only when it differs from the last kept value $a[k-1]$.

C++

```
int removeDups(vector<int>& a){
    if(a.empty()) return 0;
    int k = 1;           // write index
    for(int i = 1; i < (int)a.size(); i++)
        if(a[i] != a[k-1]) // new value
            a[k++] = a[i];
    return k;             // count of uniques
}
```

PYTHON

```
def remove_dups(a):
    if not a: return 0
    k = 1           # write index
    for i in range(1, len(a)):
        if a[i] != a[k-1]: # new value
            a[k] = a[i]; k += 1
    return k         # count of uniques
```

Complexity & notes: $O(n)$, $O(1)$. The slow/fast (write/read) two-pointer template for in-place compaction.

Q43. Left Rotate Array by One

Easy

■ **Approach:** Save the first element, shift everyone one step left, then drop the saved element at the end.

C++

```
void rotateOne(vector<int>& a){
    if(a.empty()) return;
    int first = a[0];
    for(int i = 0; i+1 < (int)a.size(); i++)
        a[i] = a[i+1];           // shift left
    a.back() = first;           // wrap around
}
```

PYTHON

```
def rotate_one(a):
    if not a: return a
    return a[1:] + a[:1]         # drop first, append it
```

Complexity & notes: $O(n)$ time, $O(1)$ space (C++ in-place). Python slicing is clean but uses $O(n)$ extra.

Q44. Left Rotate Array by D Places

Easy

■ **Approach:** The reversal trick: reverse the first d , reverse the rest, then reverse the whole array. That rotates left by d in $O(1)$ space. (Take $d \% = n$ first so $d > n$ works.)

C++

```
void rotated(vector<int>& a, int d){
    int n = a.size(); d %= n;
    reverse(a.begin(), a.begin()+d); // part 1
    reverse(a.begin()+d, a.end());   // part 2
    reverse(a.begin(), a.end());     // whole
}
```

PYTHON

```
def rotate_d(a, d):
    n = len(a); d %= n
    return a[d:] + a[:d]           # slicing version
```

Complexity & notes: $O(n)$ time. The 3-reversal method is $O(1)$ space (in-place); the slicing version is $O(n)$ space but readable.

Q45. Move Zeros to End

Easy

■ **Approach:** Slow/fast pointers: 'last' marks where the next non-zero goes. Scan with i; on each non-zero, swap it into 'last' and advance. Zeros bubble to the back, order preserved.

C++

```
void moveZeros(vector<int>& a){
    int last = 0;
    for(int i = 0; i < (int)a.size(); i++)
        if(a[i] != 0)
            swap(a[last++], a[i]);
}
```

PYTHON

```
def move_zeros(a):
    last = 0
    for i in range(len(a)):
        if a[i] != 0:
            a[last], a[i] = a[i], a[last]
            last += 1
    return a
```

Complexity & notes: O(n), O(1). Swap-based so it stays a valid permutation and preserves non-zero order in one pass.

Q46. Linear Search

Easy

■ **Approach:** Scan left to right; return the index of the first match, or -1 if not found. Works on unsorted data.

C++

```
int linearSearch(vector<int>& a, int t){
    for(int i = 0; i < (int)a.size(); i++)
        if(a[i] == t) return i;
    return -1;
}
```

PYTHON

```
def linear_search(a, t):
    for i, x in enumerate(a):
        if x == t: return i
    return -1
```

Complexity & notes: O(n), O(1). The baseline search; use binary search (O(log n)) only when the array is sorted.

Q47. Union of Two Sorted Arrays

Easy

■ **Approach:** Merge with two pointers like the merge step of merge sort. Advance the smaller side; before adding any value, skip it if it equals the last added value (keeps the union unique).

C++

```
vector<int> unionSorted(vector<int>& a, vector<int>& b){
    vector<int> r; int i=0, j=0;
    auto add=[&](int x){ if(r.empty()||r.back()!=x) r.push_back(x); };
    while(i<(int)a.size() && j<(int)b.size()){
        if(a[i] < b[j]) add(a[i++]);
        else if(a[i] > b[j]) add(b[j++]);
        else { add(a[i]); i++; j++; }
    }
    while(i<(int)a.size()) add(a[i++]);
    while(j<(int)b.size()) add(b[j++]);
    return r;
}
```

PYTHON

```
def union_sorted(a, b):
    i = j = 0; res = []
    def add(x):
        if not res or res[-1] != x: res.append(x)
    while i < len(a) and j < len(b):
        if a[i] < b[j]: add(a[i]); i += 1
        elif a[i] > b[j]: add(b[j]); j += 1
        else: add(a[i]); i += 1; j += 1
    while i < len(a): add(a[i]); i += 1
    while j < len(b): add(b[j]); j += 1
    return res
```

Complexity & notes: $O(n+m)$ time, $O(1)$ extra (besides output). Two-pointer merge — the same skill as merge sort's merge.

Q48. Intersection of Two Sorted Arrays

Easy

■ **Approach:** Two pointers again: if equal, record it and advance both; otherwise advance the pointer at the smaller value. Only shared elements get recorded.

C++

```
vector<int> intersect(vector<int>& a, vector<int>& b){
    vector<int> r; int i=0, j=0;
    while(i<(int)a.size() && j<(int)b.size()){
        if(a[i] < b[j]) i++;
        else if(a[i] > b[j]) j++;
        else { r.push_back(a[i]); i++; j++; }
    }
    return r;
}
```

PYTHON

```
def intersect(a, b):
    i = j = 0; res = []
    while i < len(a) and j < len(b):
        if a[i] < b[j]: i += 1
        elif a[i] > b[j]: j += 1
        else: res.append(a[i]); i += 1; j += 1
    return res
```

Complexity & notes: $O(n+m)$, $O(1)$. Mirror of union — advance-smaller-pointer, but only collect on equality.

Q49. Find the Missing Number

Easy

■ **Approach:** The array holds n distinct numbers from $0..n$ with exactly one missing. The full sum $0+..+n$ is $n(n+1)/2$; subtract the actual sum and what's left is the missing number. (XOR of all indices and values also works, avoiding overflow.)

C++

```
int missingNumber(vector<int>& a, int n){
    long sum = (long)n*(n+1)/2;
    for(int x : a) sum -= x;
    return sum;
}
```

PYTHON

```
def missing_number(a, n):
    return n*(n+1)//2 - sum(a)
```

Complexity & notes: $O(n)$, $O(1)$. Use long in C++ for the sum to avoid overflow. XOR variant: xor all $0..n$ and all elements.

Q50. Max Consecutive Ones

Easy

■ **Approach:** Keep a running count of consecutive 1s; reset to 0 on any 0. Track the maximum count seen.

C++

```
int maxConsecOnes(vector<int>& a){
    int mx = 0, cur = 0;
    for(int x : a){
        cur = (x == 1) ? cur+1 : 0;
        mx = max(mx, cur);
    }
    return mx;
}
```

PYTHON

```
def max_consec_ones(a):
    mx = cur = 0
    for x in a:
        cur = cur + 1 if x == 1 else 0
        mx = max(mx, cur)
    return mx
```

Complexity & notes: $O(n)$, $O(1)$. Running-count-with-reset is a mini sliding-window pattern.

Q51. Single Number (others appear twice)

Easy

■ **Approach:** XOR everything. $x \oplus x = 0$ and $x \oplus 0 = x$, so every pair cancels to 0 and only the lone number survives. No extra memory, no sorting.

C++

```
int singleNumber(vector<int>& a){
    int x = 0;
    for(int v : a) x ^= v;    // pairs cancel
    return x;
}
```

PYTHON

```
def single_number(a):
    x = 0
    for v in a: x ^= v        # pairs cancel
    return x
```

Complexity & notes: $O(n)$ time, $O(1)$ space. The XOR-cancellation trick beats the $O(n)$ -space hash-set approach.

Q52. Longest Subarray with Sum K (positives)

Easy

■ **Approach:** Since all numbers are positive, use a sliding window: expand right adding to the sum; if the sum exceeds K, shrink from the left. When the sum equals K, update the best length. Monotonic because adding is always positive.

C++

```
int longestPos(vector<int>& a, int k){
    int l = 0, best = 0; long s = 0;
    for(int r = 0; r < (int)a.size(); r++){
        s += a[r];
        while(s > k) s -= a[l++];    // shrink
        if(s == k) best = max(best, r-l+1);
    }
    return best;
}
```

PYTHON

```
def longest_pos(a, k):
    l = 0; s = 0; best = 0
    for r in range(len(a)):
        s += a[r]
        while s > k: s -= a[l]; l += 1
        if s == k: best = max(best, r-l+1)
    return best
```

Complexity & notes: $O(n)$, $O(1)$. Sliding window works ONLY because values are positive (so the sum is monotonic). For negatives, see Q53.

Q53. Longest Subarray with Sum K (positives + negatives)

Medium

■ **Approach:** With negatives the window trick breaks — use prefix sums + a hash map. For running prefix sum s at index i , a subarray summing to K ends here if some earlier prefix equals $s-K$. Store the FIRST index of each prefix to maximize length.

C++

```
int longestAny(vector<int>& a, int k){
    unordered_map<long,int> pre; long s = 0; int best = 0;
    for(int i = 0; i < (int)a.size(); i++){
        s += a[i];
        if(s == k) best = i+1;
        if(pre.count(s-k)) best = max(best, i-pre[s-k]);
        if(!pre.count(s)) pre[s] = i;    // keep earliest
    }
    return best;
}
```

PYTHON

```
def longest_any(a, k):
    pre = {}; s = 0; best = 0
    for i, x in enumerate(a):
        s += x
        if s == k: best = i+1
        if s-k in pre: best = max(best, i-pre[s-k])
        if s not in pre: pre[s] = i    # keep earliest
    return best
```

Complexity & notes: $O(n)$ time, $O(n)$ space. Prefix-sum + hashmap is THE tool for 'subarray with sum/target' when negatives are allowed.

Arrays — Medium (Q54–Q60)

Q54. Two Sum

Medium

■ **Approach:** One pass with a hash map. For each value x , check if its complement ($\text{target} - x$) was already seen; if so you've found the pair. Otherwise store x 's index. Beats the $O(n^2)$ double loop.

C++

```
vector<int> twoSum(vector<int>& a, int t){
    unordered_map<int,int> seen;
    for(int i = 0; i < (int)a.size(); i++){
        if(seen.count(t - a[i]))
            return {seen[t-a[i]], i};
        seen[a[i]] = i;
    }
    return {-1, -1};
}
```

PYTHON

```
def two_sum(a, t):
    seen = {}
    for i, x in enumerate(a):
        if t - x in seen:
            return [seen[t-x], i]
        seen[x] = i
    return [-1, -1]
```

Complexity & notes: $O(n)$ time, $O(n)$ space. If the array were sorted you could use two pointers in $O(1)$ space instead.

Q55. Sort an Array of 0s, 1s, 2s

Medium

■ **Approach:** Dutch National Flag, three pointers: low (edge of 0s), high (edge of 2s), mid (scanner). 0 -> swap to low; 1 -> leave; 2 -> swap to high (and DON'T advance mid, since the swapped-in value is unexamined). One pass.

C++

```
void sortColors(vector<int>& a){
    int lo=0, mid=0, hi=a.size()-1;
    while(mid <= hi){
        if(a[mid]==0) swap(a[lo++], a[mid++]);
        else if(a[mid]==1) mid++;
        else swap(a[mid], a[hi--]); // no mid++
    }
}
```

PYTHON

```
def sort_colors(a):
    lo = mid = 0; hi = len(a)-1
    while mid <= hi:
        if a[mid]==0: a[lo],a[mid]=a[mid],a[lo]; lo+=1; mid+=1
        elif a[mid]==1: mid+=1
        else: a[mid],a[hi]=a[hi],a[mid]; hi-=1
    return a
```

Complexity & notes: $O(n)$, $O(1)$, single pass. The 'don't advance mid after a 2-swap' detail is the classic interview trap.

Q56. Majority Element (> n/2)

Medium

■ **Approach:** Boyer-Moore voting: keep a candidate and a count. count 0 -> adopt current as candidate. Same as candidate -> count+1, else count-1. The majority can't be fully cancelled (it appears > half), so it survives.

C++

```
int majority(vector<int>& a){
    int cand = 0, cnt = 0;
    for(int x : a){
        if(cnt == 0) cand = x;
        cnt += (x == cand) ? 1 : -1;
    }
    return cand;
}
```

PYTHON

```
def majority(a):
    cand = None; cnt = 0
    for x in a:
        if cnt == 0: cand = x
        cnt += 1 if x == cand else -1
    return cand
```

Complexity & notes: O(n) time, O(1) space. Beats the hash-count (O(n) space) and sort-and-take-middle (O(n log n)) approaches.

Q57. Kadane's Algorithm (Max Subarray Sum)

Medium

■ **Approach:** At each element decide: extend the current running sum, or start fresh from this element (whichever is larger). Track the best running sum ever seen. Starting fresh whenever the running sum would drag you down is the key insight.

C++

```
int kadane(vector<int>& a){
    int best = a[0], cur = a[0];
    for(int i = 1; i < (int)a.size(); i++){
        cur = max(a[i], cur + a[i]); // extend or restart
        best = max(best, cur);
    }
    return best;
}
```

PYTHON

```
def kadane(a):
    best = cur = a[0]
    for x in a[1:]:
        cur = max(x, cur + x) # extend or restart
        best = max(best, cur)
    return best
```

Complexity & notes: $O(n)$, $O(1)$. One of the most-asked array problems. The 'extend vs restart' decision is the whole trick.

Q58. Print the Max-Sum Subarray

Medium

■ **Approach:** Kadane, but also record WHERE the best window is. When you restart (cur resets to $a[i]$), mark a new tentative start. Whenever cur beats best, lock in start..i as the current best window.

C++

```
vector<int> kadaneRange(vector<int>& a){
    int best=a[0], cur=a[0], s=0, bs=0, be=0;
    for(int i=1;i<(int)a.size();i++){
        if(cur + a[i] < a[i]){ cur=a[i]; s=i; } // restart
        else cur += a[i];
        if(cur > best){ best=cur; bs=s; be=i; } // lock window
    }
    return vector<int>(a.begin()+bs, a.begin()+be+1);
}
```

PYTHON

```
def kadane_range(a):
    best=cur=a[0]; s=bs=be=0
    for i in range(1, len(a)):
        if cur + a[i] < a[i]: cur=a[i]; s=i # restart
        else: cur += a[i]
        if cur > best: best=cur; bs=s; be=i # lock window
    return a[bs:be+1]
```

Complexity & notes: $O(n)$, $O(1)$. Same Kadane loop; the only extra work is tracking the start (s) and best window (bs..be).

Q59. Best Time to Buy and Sell Stock

Medium

■ **Approach:** Track the cheapest price seen so far. At each day, the best sale-today profit is price - min_so_far; keep the maximum of those. One pass — no need to try every buy/sell pair.

C++

```
int maxProfit(vector<int>& p){
    int lo = INT_MAX, best = 0;
    for(int x : p){
        lo = min(lo, x);           // cheapest buy
        best = max(best, x - lo);  // best sale today
    }
    return best;
}
```

PYTHON

```
def max_profit(p):
    lo = float("inf"); best = 0
    for x in p:
        lo = min(lo, x)           # cheapest buy
        best = max(best, x - lo)  # best sale today
    return best
```

Complexity & notes: $O(n)$, $O(1)$. The 'track the best thing so far in one pass' pattern (here the min price). Return 0 if no profit.

Q60. Rearrange Array by Sign

Medium

■ **Approach:** Equal positives and negatives; place positives at even indices (0,2,4,...) and negatives at odd (1,3,5,...), preserving their relative order. Use a result array with two write pointers, one stepping by 2 from 0, one from 1.

C++

```
vector<int> rearrange(vector<int>& a){
    vector<int> r(a.size());
    int pi = 0, ni = 1;
    for(int x : a){
        if(x > 0){ r[pi] = x; pi += 2; }
        else { r[ni] = x; ni += 2; }
    }
    return r;
}
```

PYTHON

```
def rearrange(a):
    r = [0]*len(a); pi = 0; ni = 1
    for x in a:
        if x > 0: r[pi] = x; pi += 2
        else: r[ni] = x; ni += 2
    return r
```

Complexity & notes: $O(n)$ time, $O(n)$ space. Two write-pointers stepping by 2 keep positives and negatives interleaved in order.

That's Q41–60 — the core Arrays toolkit. The patterns you now own: **two pointers** (dedup, union, intersection, move-zeros), **sliding window** (sum K positives, consecutive ones), **prefix-sum + hashmap** (sum K with negatives), **XOR tricks** (missing number, single number), and the FAANG favourites — **Two Sum**, **Dutch flag**, **Boyer–Moore**, **Kadane**, **Buy/Sell Stock**. These five alone show up in a huge share of interviews. Re-type each from a blank page in both languages. Next: Q61–80 (the harder array problems + start of Binary Search).